Final Report

CMS 375

Caitlyn Lefebvre, Stella Fruijtier, Sydney Eckstein

Introduction

Balancing the dual responsibilities of academics and athletics presents a unique challenge for college student-athletes. With demanding practice schedules, competitions, meetings, and coursework, many rely on a fragmented combination of personal calendars, messaging applications, spreadsheets, and verbal communication to stay organized. This disconnected approach often leads to scheduling conflicts, missed updates, communication gaps, and unnecessary stress that can negatively impact both athletic performance and academic success.

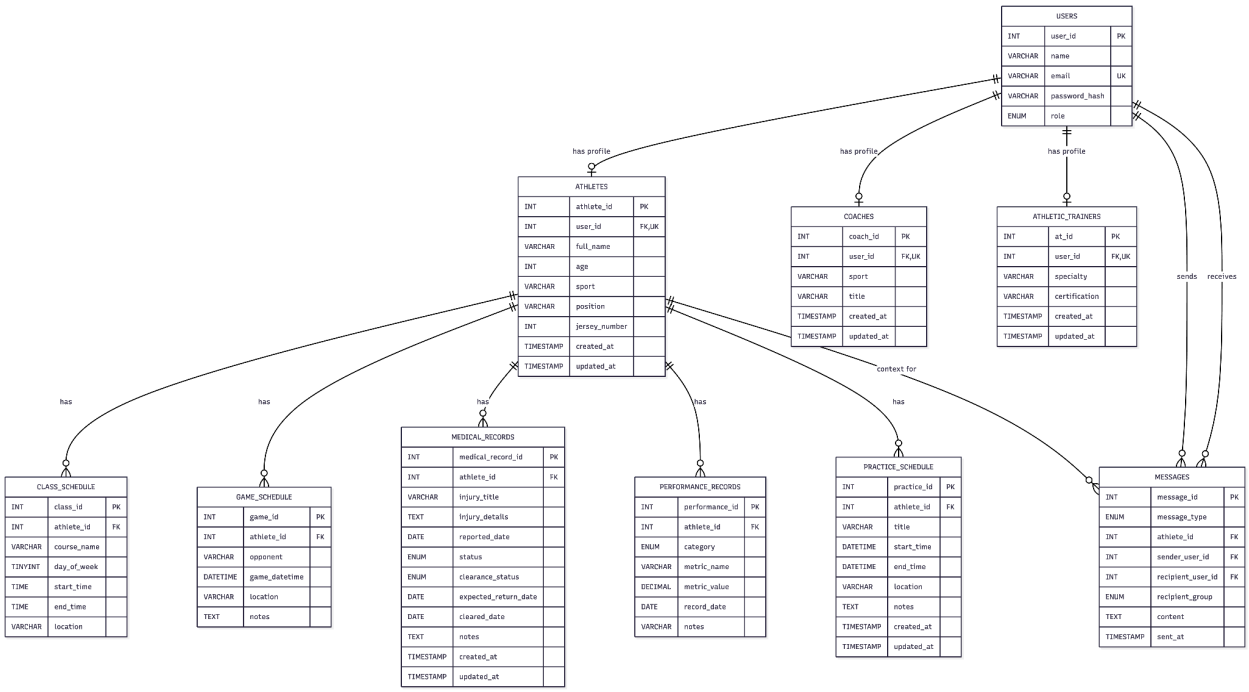
To address these challenges, our team developed AthleteHub, a centralized web-based athlete management platform designed specifically for college athletic programs. The purpose of the system is to combine scheduling, communication, athlete monitoring, and profile management into one integrated environment that is simple, secure, and role-specific.

The platform provides separate dashboards for three primary user groups:

- Athletes: view schedules, timetables, communicate with staff, track performance, and access medical updates.
- Coaches: manage practices and matches, communicate with athletes and staff, and monitor athlete availability.
- Athletic Trainer: manage medical records, communicate with athletes and coaches, and monitor scheduling needs.

ER Diagram

The following ER Diagram illustrates the relational structure of AthleteHub. It shows the core entities of the system, including users, athletes, coaches, athletic trainers, schedules, medical records, performance records, and messaging. The diagram also highlights the one-to-one and one-to-many relationships used to support role-based profiles and operational functionality.



Database Design

AthleteHub was developed using MySQL. The database was structured to ensure consistency, scalability, and efficient retrieval of information across multiple user roles. Instead of storing all data in a single table, the system separates information into specialized tables connected through primary and foreign keys.

Schema: (PK underlined, FK italicized)

- users(<u>user_id</u>, name, email, password_hash, role)
- athletes(<u>athlete_id</u>, *user_id*, full_name, age, sport, position, jersey_number, created_at, updated_at)
- coaches(<u>coach_id</u>, *user_id*, sport, title, created_at, updated_at)
- athletic_Trainers(<u>at_id</u>, *user_id*, specialty, certification, title, created_at, updated_at)
- practice_Schedule(<u>practice_id</u>, *athlete_id*, title, start_time, end_time, location, notes, created_at, updated_at)
- game_Schedule(<u>game_id</u>, *athlete_id*, opponent, game_datetime, location, notes)
- class_Schedule(<u>class_id</u>, *athlete_id*, course_name, day_of_week, start_time, end_time, location)
- performance_Records(<u>performance_id</u>, *athlete_id*, category, metric_name, metric_value, record_date, notes)
- medical_Records(<u>medical_record_id</u>, *athlete_id*, injury_title, injury_details, reported_date, status, clearance_status, expected_return_date, cleared_date, notes, created_at, updated_at)
- messages(<u>message_id</u>, message_type, *athlete_id, sender_user_id, recipient_user_id*, recipient_group, content, sent_at)

Core table: users, serves as the foundation of the system and stores authentication data shared by all account types.
It includes:

- user_id
- name
- email
- password_hash
- role (athlete, coach, athletic_trainer)

This design allows one centralized login system while supporting multiple user roles.

Profile extension tables:

To avoid redundancy, role-specific information is stored in separate tables:

- Athletes: athlete profile details such as sport, position, age, jersey number, and full name
- Coaches: coach-specific information such as sport and title
- Athletic trainers: trainer information such as specialty and certification

Each of these tables references users.user_id with a one-to-one relationship.

Operational tables:

practice_schedule: stores scheduled training sessions assigned to athletes.

game_schedule: stores upcoming matches, opponents, dates, and locations.

class_schedule: stores recurring weekly academic schedules for athletes using day_of_week, start_time, and end_time. This reduces duplicated semester data.

medical_records: stores injury reports, recovery status, clearance status, expected return dates, and notes.

performance_records: stores athletic metrics and progress records used for performance monitoring.

messages: stores both direct messages and team announcements through one flexible structure.

Design decisions:

Several important database decisions were made during development:

- Normalization: shared account data was separated from role-specific tables.
- Foreign keys: used to maintain valid relationships and prevent orphaned records
- Prepared statements: used in PHP to improve security and reduce SQL injection risk.

- Role-Based Schema: different user roles interact with the same logic through controlled permissions.
- Scalability: new roles or additional scheduling modules can be added later with minimal redesign.

Implementation Details

Database Design: Created nine relational databases which are all linked through foreign keys. Below are code snippets of the athlete and users relational tables, where the others follow a similar format with their respective attributes.

```sql
-- Athletes table
USE athletics_db;

CREATE TABLE IF NOT EXISTS athletes (
  athlete_id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  full_name VARCHAR(100) NOT NULL,
  age INT NULL,
  sport VARCHAR(50) NULL,
  position VARCHAR(50) NULL,
  jersey_number INT NULL,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT fk_athletes_user
    FOREIGN KEY (user_id) REFERENCES users(user_id)
    ON DELETE CASCADE
    ON UPDATE CASCADE,
  CONSTRAINT uq_athletes_user UNIQUE (user_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;


SELECT * FROM users;
SELECT * FROM athletes;
```

```sql
-- Users table


USE athletics_db;

CREATE TABLE IF NOT EXISTS users (
  user_id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  email VARCHAR(255) NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  role ENUM('athlete','coach','athletic_trainer') NOT NULL,
  CONSTRAINT uq_users_email UNIQUE (email)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

SELECT user_id, name, email, role FROM users;
```

Authentication and Role Routing: Users register with a role and passwords are hashed using PHP's built-in password_hash() function.

```php
$hashedPassword = password_hash($password, PASSWORD_DEFAULT);

$_SESSION['role'] = $user['role'];


if ($role === 'coach') {
   header("Location: coach_dashboard.php");
} elseif ($role === 'athletic_trainer') {
   header("Location: at_dashboard.php");
} else {
   header("Location: index.php");
}
```

Role-based Access Control: Checks the session role at the top to route the user to the correct dashboard.

```
$role = strtolower(trim($_SESSION['role']));
if ($role !== 'coach') {
    header("Location: index.php");
    exit();
}
```

Database Connection: All files share a single db.php connection file.

```
$conn = new mysqli($host, $username, $password, $dbname);
```

Form Handling: Each form communicates with a dedicated handler that validates input.

```
$stmt = $conn->prepare(
    "INSERT INTO medical_records
     (athlete_id, injury_title, reported_date, status, clearance_status)
     VALUES (?, ?, ?, ?, ?)"
);
$stmt->bind_param("issss", $athleteId, $injuryTitle, $reportedDate, $status, $clearanceStatus);
$stmt->execute();
```

Communication System: A messages table is used to handle both direct messages and team announcements through a message_type column.

```
// Team announcement
"INSERT INTO messages (message_type, sender_user_id, recipient_group, content)
 VALUES ('announcement', ?, 'team', ?)"

// Direct message
"INSERT INTO messages (message_type, athlete_id, sender_user_id, recipient_user_id, content)
 VALUES ('direct', ?, ?, ?, ?)"
```

Dynamic Dashboards: each dashboard queries live data and renders it with php.

```
$stmt = $conn->prepare(
    "SELECT injury_title, status, clearance_status
     FROM medical_records
     WHERE athlete_id = ?
     ORDER BY reported_date DESC"
```

```
);
$stmt->bind_param("i", $athleteId);
$stmt->execute();
$records = $stmt->get_result();
```

Stat Summary Cards: Every dashboard calculates live counts from query results.

```
$activeInjuries = count(array_filter($medicalRecords,
    fn($r) => $r['status'] === 'active'));
$recovering = count(array_filter($medicalRecords,
    fn($r) => $r['status'] === 'recovering'));
```

Challenges

One challenge we encountered was with connectivity between different user profiles. This applies to components such as posting messages, medical updates, practice schedules, etc. We had difficulty with messages and added practices not appearing on the receiver's end. Solving this included a meticulous process in the backend as we first had to ensure that each form was submitting properly and subsequently had to map out the appropriate page to be updated and displayed once that action had been taken. We had found that some forms were not being fed through across the profiles properly which led to this issue and got it fixed.

Secondly, we struggled with the appropriate layout of the sidebar containing the numerous pages that users could utilize to move between different pages (performance, medical history, etc). As this was a major UI component in our database, it was necessary that this was functional and was aesthetically pleasing to the user. Initially, the sidebar was not on the left side but rather pushed to the bottom of the page due to other containers being stacked. This was fixed using "display: flex" which was added to the CSS file so that the sidebar and main content are side by side horizontally.

Lastly, another main issue we encountered was with role-based access control. We wanted to ensure that our project provided a different experience for each profile (player, coach, and athletic trainer) as each role is only interested in features that apply to them and not their counterparts. To achieve this, we implemented role-based routing with separate handler files to automatically direct users to their appropriate dashboard upon login by storing their role in the PHP session. Each page validates their role which also redirects unauthorized users away immediately.

Future Work

Future development of this project would focus on enhancing usability, accessibility, and overall functionality to better support student-athletes and coaching staff. One key improvement is the implementation of a notification and alert system to provide real-time updates for schedule changes, upcoming events, and important announcements, reducing the likelihood of missed commitments. Expanding communication features—such as group messaging, read receipts, and more interactive channels—would further streamline team coordination. Additionally, a performance analytics dashboard could be introduced to visualize athlete progress through detailed statistics and trends, offering valuable insights for athletes, coaches, and athletic trainers. Finally, developing a mobile version of the platform would significantly increase accessibility, allowing users to manage their schedules and stay connected on the go. Together, these enhancements would build upon the current system to create a more comprehensive and user-friendly experience.